UNIVERSITI TUNKU ABDUL RAHMAN

FACULTY OF INFORMATION & COMMUNICATION TECHNOLOGY

BACHELOR OF INFORMATION TECHNOLOGY (HONS)
COMPUTER ENGINEERING

UCCE3033 EMBEDDED SYSTEMS DESIGN

TITLE: 2-in-1 Instruments

No.	Name	Student ID	Part
1.	Goh Xuan Hui	2301839	Voltage, resistor, connectivity
2.	Wong Carman	2305293	Temperature, capacitance, current
3.	Elveis Ng Kai Wei	2204851	Oscillator, UI

TABLE CONTENT

1.0 General Information

1.1 Introduction

This project involves the design and implementation of a 2-in-1 educational measurement instrument using an STM32 microcontroller. The device combines the functionality of both a Digital Multimeter (DMM) and a Dual-Channel Oscilloscope into a single, compact and portable unit. The goal is to create a practical tool that can measure key electrical parameters such as voltage, current, capacitance, resistance and frequency, while also providing users with the ability to capture, visualize and analyze electronic signals through real-time waveform display. By integrating both measurement tools into one device, this project offers an efficient solution for circuit testing and troubleshooting.

1.2 Objective

The objective of this project is to design and develop a 2-in-1 measurement instrument that combines the functionality of a Digital Multimeter (DMM) and a Dual-Channel Oscilloscope using an STM32 microcontroller. The device aims to provide an efficient, portable solution for a range of electronic measurements and signal analysis.

- To develop a functional dual-channel oscilloscope capable of displaying signals on an LCD with adjustable time bases and measurement features.
- To enable system integration that allows users to seamlessly switch between oscilloscope and DMM modes.
- Enable system integration so that users can seamlessly switch between oscilloscope and DMM modes.

2.0 Product Specifications

2.1 Oscilloscope

2.1.1 Functionality/Feature

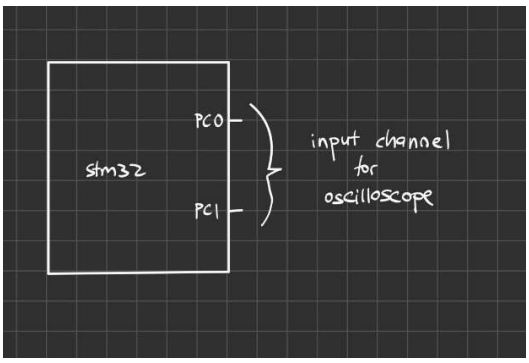
Core Features:

- Dual-channel oscilloscope with simultaneous sampling of CH1 (PC0) and CH2 (PC1).
- Adjustable time base with 7 settings (1s/div, 500ms/div, 250ms/div, 100ms/div, 50ms/div, 10ms/div, 5ms/div).
- Real-time waveform display on 320 x 240 LCD with grid.
- Channel control, allow independent enable / disable for CH1 and CH2.
- Waveform measurement includes Frequency calculation, Average Voltage, Duty Cycle percentage, Peak-to-peak Voltage.
- Waveform storage, allow store and recall functionality using NAND flash.

Display Features:

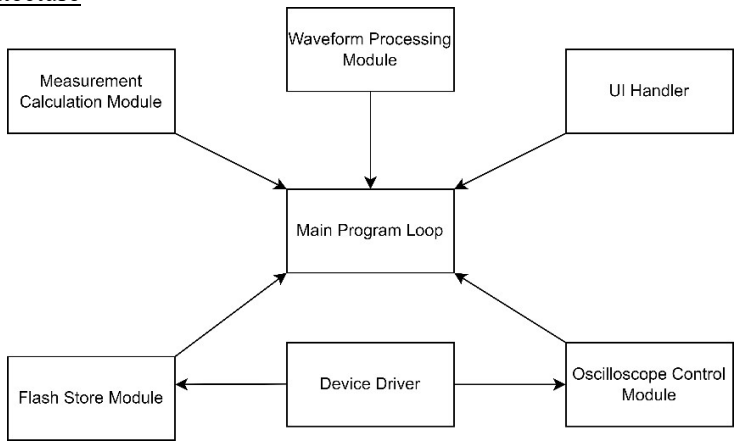
- 6 x 4 division grid (270 x 172 pixels waveform display area).
- Yellow trace for CH1, Cyan trace for CH2.
- Real-time Measurement display above grid.

2.1.2 Hardware Architecture

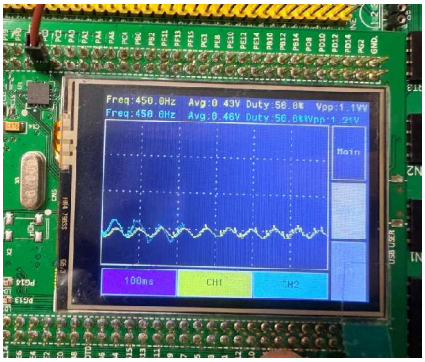


- ADC: 12-bits ADC sampling
- DMA Controller: Direct Memory Access for efficient data transfer from ADC to memory.
- Timer Modules: TIM2 for generate ADC sampling triggers, TIM3 control display refresh rate 33Hz
- NAND flash: 2KB/page storage for waveform data preservation
- Analog signals enter through pc0 (CH1) and PC1 (CH2)

2.1.3 Software Architecture



2.1.4 User Interface



This figure shows the oscilloscope display on the TFT screen of the STM32 development board.

2.1.5 Algorithms

1) Sampling
    - Timer (TIM2) triggers ADC conservation for two channels.
    - ADC results are transferred into memory using DMA.
2) Data Processing
    - Collected samples are separated into CH1 and CH2.
    - Calculate measurements: Vpp, Avg, Duty Cycle, Frequency.
3) Display
    - Clear and redraw grid on LCD
    - Plot waveforms for CH1 and CH2 with different colors.
    - Display calculated values on LCD
4) Waveform Storage
    - Optionally save current waveform data into NAND flash
    - Recall saved waveform for display.

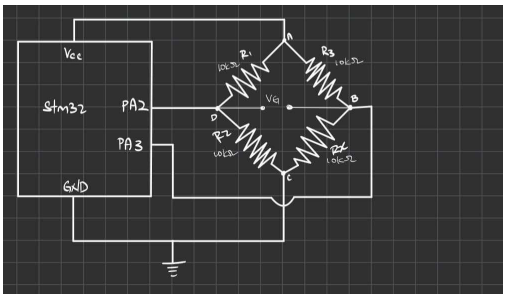
2.2 Digital Multimeter (DMM)

2.2.1 Voltage measurement (DC input via ADC).

2.2.1.1 Functionality/Feature

- Measure DC voltage from external input connected to PA0 and PA1.
- Display DC voltage result in mV on LCD screen

2.2.1.2 Hardware Architecture



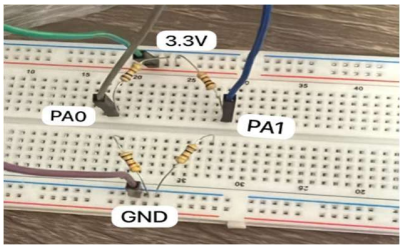
This figure shows the connection between an STM32 and Wheatstone Bridge circuit.

The voltage measurement for the Digital Multimeter (DMM) is implemented using a Wheatstone Bridge circuit connected to the STM32 microcontroller. The bridge consists of four resistors: R1, R2, R3, and Rx. In this design:

- R1=R2=R3=10kΩ (fixed resistors).
- Rx is also set as 10kΩ for testing purposes.
- The bridge is powered by a 3.3V supply.
- The intermediate voltages are taken as:
    - V_a : at the junction between R1 and R2, connected to PA0 (ADC1_CH0).
    - V_b : at the junction between R3 and Rx, connected to PA1 (ADC1_CH1).
- The common node is tied to GND.
- The STM32 ADC (12-bit resolution) is used to digitize V_a and V_b .

This allows the microcontroller to compute the voltage difference:

$VG = VD - VB$



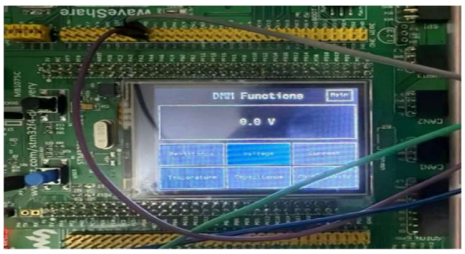
This image shows the physical connection of the Wheatstone Bridge with the necessary components.

2.2.1.3 Software Architecture (Pseudocode)

```
// Step 1: Check if the correct button (DMM Voltage) is pressed

if button_id equals dmm_vol_button_id then

// Step 2: De-initialize previous DMM modes to clear settings
Call DMM_CAP_Deinit()   // De-initialize capacitance measurement
Call DMM_VOL_Deinit()   // De-initialize voltage measurement
Call DMM_TEMP_Deinit()  // De-initialize temperature measurement
Call DMM_RES_Deinit()   // De-initialize resistance measurement

// Step 3: Set the system into voltage measurement mode
Set cur_mode to DMM_MODE          // Change the current mode to Digital Multimeter (DMM) mode
Set DMM_BUTTON_SEL to DMM_VOL_MODE  // Set the active button selection to voltage mode

// Step 4: Declare variables to store voltage and current values
Declare Va_mV as float = 0.0   // Voltage at point A in millivolts
Declare Vb_mV as float = 0.0   // Voltage at point B in millivolts
Declare Vrslt as float = 0.0   // Resulting voltage difference (Va - Vb)
Declare current_mA as float = 0.0  // Current in milliamps

// Step 5: Configure the ADC for voltage/current measurement
Call DMM_VOL_Config()  // Configure ADC to read PA0 (Va) and PA1 (Vb)

// Step 6: Perform measurement
Call VoltageMeasure(&Va_mV, &Vb_mV, &current_mA)
   // Measure voltage at point A (Va)
   // Measure voltage at point B (Vb)
   // Calculate current using shunt resistor Rs

// Step 7: Update button highlights without redrawing entire display
Call DrawButtons()  // Update the button highlights on the screen

// Step 8: Calculate voltage difference
Set Vrslt = Va_mV - Vb_mV
```

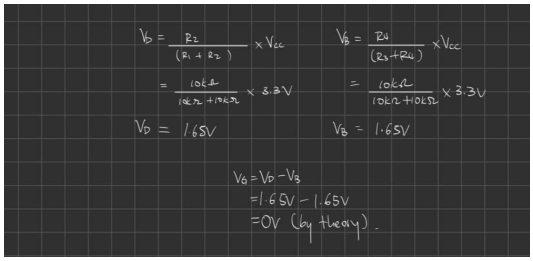
```
// Step 9: Format the voltage result into string
Call float_to_str(Vrslt, temp, 2)
Call snprintf(str, sizeof(str), "%s mV", temp)

// Step 10: Display the voltage result on the LCD
Call DisplayCenteredTextInRectangle(
    DMM_DISPLAY_X,      // X position of display area
    DMM_DISPLAY_Y,      // Y position of display area
    DMM_DISPLAY_WIDTH,  // Width of display area
    DMM_DISPLAY_HEIGHT, // Height of display area
    str,                // The formatted voltage string to display
    &TM_Font_11x18,     // Font style
    ILI9341_COLOR_WHITE // Text color (white)
)  end if // End of button press check
```

2.2.1.4 User Interface



This image shows the display on the LCD when the voltage button is pressed and the voltage value is successfully read and displayed correctly.

2.2.1.5 Algorithm



This figure shows the theory calculation of the voltage using voltage divider rules.

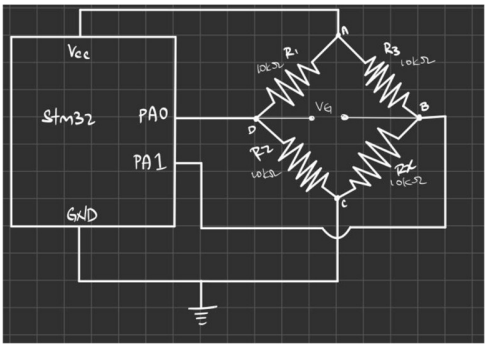
1. Read voltages from two input points (V_a and V_b) using the ADC.
2. Convert ADC values to actual voltage by scaling with reference voltage (3.3 V) and resolution (12-bit → 4095).
3. Calculate voltage difference: $V_{result} = V_a - V_b$
4. Display the voltage difference on the LCD in millivolts (mV).

2.2.2 Current measurement (Wheatstone bridge method).

2.2.2.1 Functionality/Feature

- Measure net DC current from input connected to PA2 and PA3.
- Display current result in mA on LCD screen.

2.2.2.2 Hardware Architecture



This figure shows the design of a current measurement circuit.

1. Microcontroller (STM32, Pins PA0 and PA1):

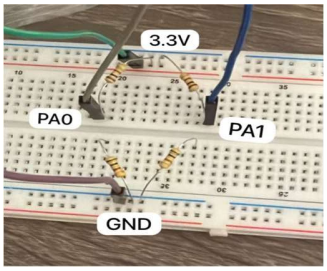
   - Pin PA0 is connected to node D (left midpoint of the bridge).
   - Pin PA1 is connected to node B (right midpoint of the bridge).
   - Both pins are configured as ADC inputs to acquire the corresponding voltages.

2. Resistors (R1, R2, R3, R4 = 10 kΩ):

   - The four resistors form the arms of the Wheatstone bridge.
   - With equal resistance values, the midpoint voltages are each 1.65 V when powered from a 3.3 V supply, resulting in a balanced condition.

3. Power Supply (Vcc = 3.3 V, GND):

   - The bridge is powered directly by the STM32 microcontroller's supply voltage.
   - The top node of the bridge is connected to Vcc, and the bottom node is tied to ground.



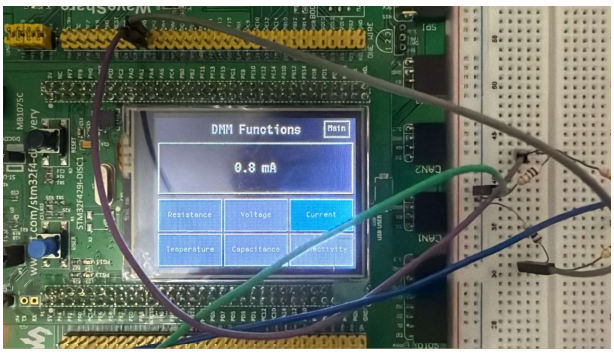
This image shows a breadboard setup for measuring current using the STM32 microcontroller's PA0 and PA1 pins.

2.2.2.3 Software Architecture (Pseudocode)

```
IF button is pressed AND button_id equals DMM current button ID THEN:

  // Step 1: Deinitialize all other modes to prepare for current measurement mode
  Deinitialize DMM capacitance mode (DMM_CAP_Deint)
  Deinitialize DMM voltage mode (DMM_VOL_Deint)
  Deinitialize DMM temperature mode (DMM_TEMP_Deint)
```

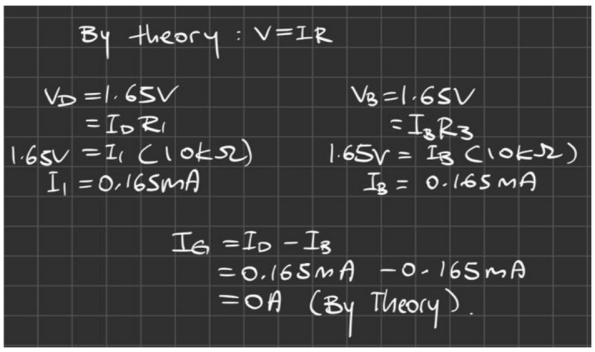
```
    Deinitialize DMM resistance mode (DMM_RES_Deint)

    // Step 2: Set the mode to current measurement mode
    Set current measurement mode as active (cur_mode = DMM_MODE)

    // Step 3: Set the current button selection to the "current mode"
    Set DMM_BUTTON_SEL to DMM_CUR_MODE

    // Step 4: Initialize measurement variables for voltage and current
    Initialize voltage values Va and Vb to 0.0 (float Va = 0.0, Vb = 0.0)
    Initialize current_mA to 0.0 (float current_mA = 0.0)

    // Step 5: Configure the voltage measurement system
    Call DMM_VOL_Config() to configure voltage measurement settings

    // Step 6: Perform voltage and current measurement
    Call VoltageMeasure(&Va, &Vb, &current_mA) to measure the voltage and current
    // The measured voltage values will be stored in Va and Vb
    // The measured current value will be stored in current_mA

    // Step 7: Update the button highlights on the screen
    Call DrawButtons() to update the appearance of the buttons, highlighting the "current" button

    // Step 8: Convert the measured current value (current_mA) into a string
    Call float_to_str(current_mA, temp, 2) to convert current_mA to a string with 2 decimal places
    // This will store the formatted string in 'temp'

    // Step 9: Format the string for display
    Use snprintf() to create a formatted string with the current value in mA:
    snprintf(str, sizeof(str), "%s mA", temp)
    // The formatted string (e.g., "0.8 mA") will be stored in 'str'

    // Step 10: Display the formatted current value in the center of the designated display area
    Call DisplayCenteredTextInRectangle(DMM_DISPLAY_X, DMM_DISPLAY_Y,
                                        DMM_DISPLAY_WIDTH, DMM_DISPLAY_HEIGHT,
                                        str, &TM_Font_11x18, ILI9341_COLOR_WHITE)
    // This will display the current measurement in the specified rectangle with white text on the screen

END IF
```

2.2.2.4 User Interface



This figure shows the display when the user selects the "Current" function showing a current measurement of 0.8 mA.

2.2.2.5 Algorithm



This figure shows the theoretical calculation of the net currentI_G.

By applying Ohm's Law(V=IR), the current can be calculated through each resistor based on the voltage drop across them. This theoretical calculation shows that the net current I_G is 0A because the currents I_D and I_B are equal and cancel each other out.

In the design, the measured current is 0.8A, which deviates from the theoretical value of 0A. The slight discrepancy between the theoretical value and the output value displayed on the LCD can be attributed to various factors that affect the accuracy and precision of the measurement. These discrepancies could arise from issues such as

1. Measurement Tolerances of Resistors:

- Resistor Tolerances: Resistors have manufacturing tolerances (e.g., ±1%, ±5%) which means the actual resistance value could be slightly different from the nominal value (like 10kΩ or 0.1Ω). This would affect the voltage drop and, subsequently, the current calculation.

2. Temperature Variations:

- Temperature Effects: Resistor values can change with temperature. If your circuit heats up during operation, resistors like the shunt resistor may change in value, leading to inaccurate current readings.

3. Power Supply Noise or Instability:

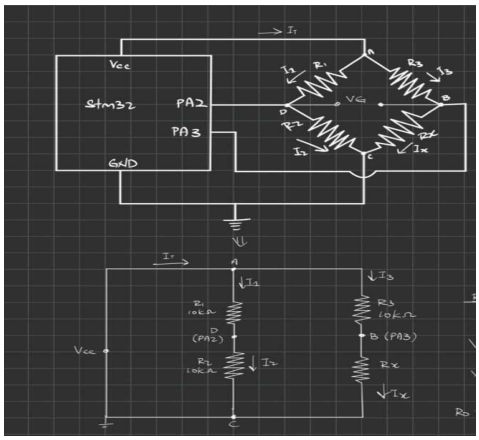
- Noise in Power Supply: If there is noise or instability in the power supply (Vcc), this can affect the accuracy of voltage measurements, especially for small signals like those across a shunt resistor.

2.2.3 Resistance measurement (Wheatstone bridge).

2.2.3.1 Functionality/Feature

- Determine an unknown resistor Rx using a Wheatstone Bridge configuration.
- STM32 reads the voltages at two midpoints of the bridge ,V_B and V_A through its ADC channels.
- The system calculates the value of Rx and displayed result on the LCD screen in ohms(Ω).

2.2.3.2 Hardware Architecture



This figure shows the connection between STM32 and Wheatstone Bridge circuit and theory calculation.

Wheatstone Bridge Configuration:

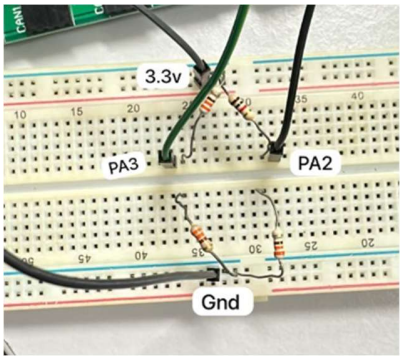
- Four resistors form the bridge: R1, R2, R3 (10k each) and Rx (unknown).
- The bridge is powered by *Vs*=3.3V.

Voltage Measurement Points:

- Junction of R1 and R2 connected to STM32 ADC1 Channel3 (PA3).
- Junction of R3 and Rx connected to STM32 ADC1 Channel2 (PA2).

STM32 Microcontroller:

- Computes the voltage difference VG =VD −VB.



This image shows the physical connection of the Wheatstone Bridge with the necessary components.

2.2.3.3 Software Architecture (Pseudocode)

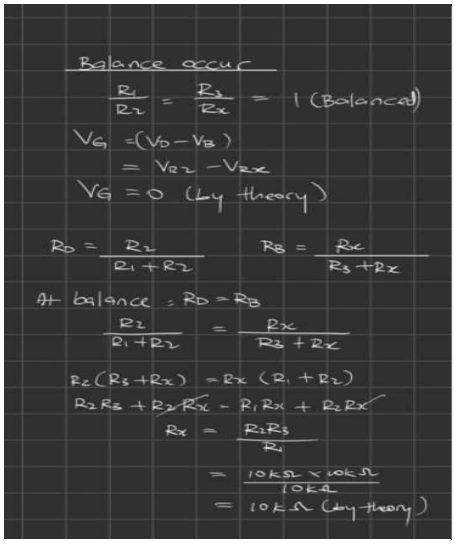
```
// Step 1: Check if the correct button (DMM Resistance) is pressed

if button_id equals dmm_res_button_id then

// Step 2: De-initialize previous DMM modes to clear settings
Call DMM_CAP_Deinit()   // De-initialize capacitance measurement
Call DMM_VOL_Deinit()   // De-initialize voltage measurement
Call DMM_TEMP_Deinit()  // De-initialize temperature measurement
Call DMM_RES_Deinit()   // De-initialize resistance measurement (reset before reusing)

// Step 3: Set the system into resistance measurement mode
Set cur_mode to DMM_MODE            // Change the current mode to Digital Multimeter (DMM) mode
Set DMM_BUTTON_SEL to DMM_RES_MODE  // Set the active button selection to resistance mode

// Step 4: Initialize the hardware for resistance measurement
Call DMM_RES_Config()  // Configure ADC channels and GPIO for Wheatstone bridge input

// Step 5: Declare variable to store resistance value
Declare read_value as float  // Variable to hold the measured resistance

// Step 6: Perform resistance measurement
Set read_value = Call resisMeasure()
   // Reads voltages from Wheatstone bridge nodes
   // Calculates unknown resistor Rx based on bridge equation

// Step 7: Update button highlights without redrawing entire display
Call DrawButtons()  // Update the button highlights on the screen

// Step 8: Format resistance value into string
Call float_to_str(read_value, temp, 2)
Call snprintf(str, sizeof(str), "%s Ohm", temp)

// Step 9: Display the resistance result on the LCD
Call DisplayCenteredTextInRectangle(
```

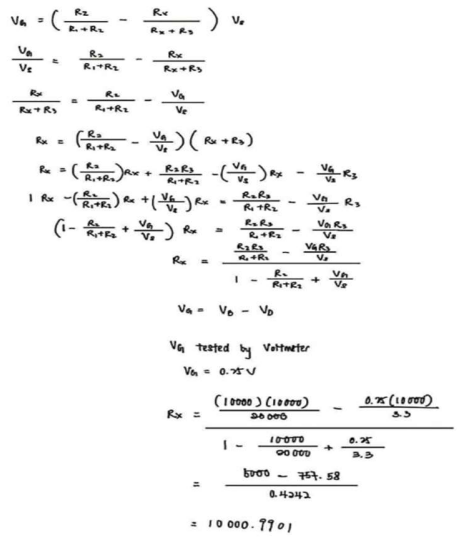
```
    DMM_DISPLAY_X,                  // X position of display area
    DMM_DISPLAY_Y,                  // Y position of display area
    DMM_DISPLAY_WIDTH,              // Width of display area
    DMM_DISPLAY_HEIGHT,             // Height of display area
    str,                            // The formatted resistance string
    &TM_Font_11x18,                 // Font style
    ILI9341_COLOR_WHITE             // Text color (white)
)
 end if // End of button press check
```

2.2.3.4 User Interface

This image shows the display on the LCD when the Resistance button is pressed, and the Rx value successfully displayed.

2.2.3.5 Algorithm





These two figures show the theory and practical calculation of Wheatstone Bridge circuit.

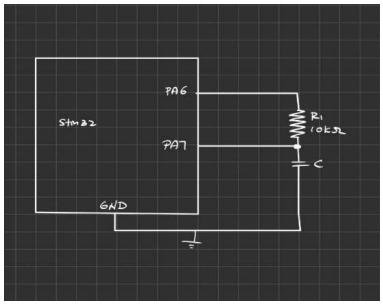
1. Read voltages from the Wheatstone bridge nodes (V_B and V_D) using the ADC.
2. Calculate bridge output voltage: VG =VB −VD
3. Apply Wheatstone bridge equation to solve for unknown resistance Rx:
   Rx = ((((VS * R2) - (VG * (R1 + R2))) / ((VS * R1) - (VG * (R1 + R2))))) * R3
4. Return and display the resistance value on the LCD in ohms (Ω).

2.2.4 Capacitance measurement (charge/discharge method).

2.2.4.1 Functionality / Feature

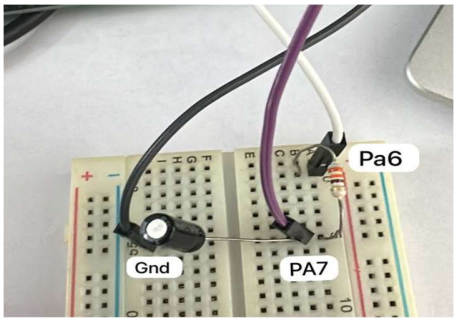
- Measure capacitance by charging and discharging a capacitor.
- Display the capacitance value in uF or mF depending on the magnitude.
- Automatically reset for the next measurement after displaying the result.

2.2.4.2 Hardware Architecture



This figure shows the design of a capacitance measurement circuit.

- Microcontroller: STM32 microcontroller, with the data line connected to PA6 and PA7 pins.
- Capacitor: The capacitor (C) whose capacitance is to be measured.
- Resistor: A 10kΩ resistor placed in series with the capacitor to control the charge/discharge rate for accurate capacitance measurement.
- Power Supply: The STM32 is powered via the VCC pin (3.3V).
- Grounding: Connect the STM32 to ground (GND) to complete the circuit.



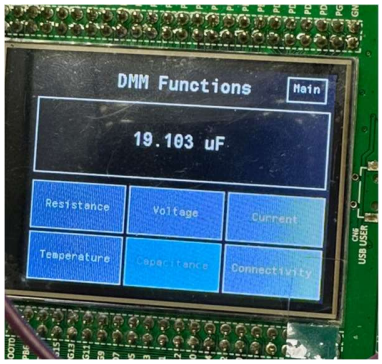
This image shows a breadboard setup for measuring capacitance.

2.2.4.3 Software architecture (pseudocode)

```
// Check if the user presses the capacitance button
IF button_id is equal to dmm_cap_button_id THEN
    // Switch to capacitance measurement mode
    Set cur_mode to DMM_MODE
    Set DMM_BUTTON_SEL to DMM_CAP_MODE
    Call DMM_CAP_Config()

    // Initialize measurement settings
    Set cap_measurement_active to 1
    Set capState to CAP_STATE_DISCHARGED
```

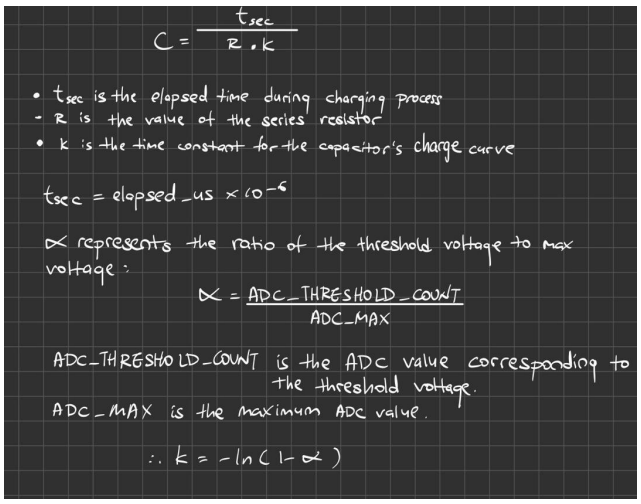
```
    Set cap_measurement_done to 0
    Set capacitance_value to 0.0f
    Set cap_start_time to TIM2->CNT
END IF

// Perform capacitance measurement if active
IF cap_measurement_active is true THEN
    Call ProcessCapacitanceMeasurement()

    IF cap_measurement_done is true THEN
        Set cap_measurement_done to 0
        Call TM_ILI9341_DrawFilledRectangle()  // Clear display area

        Call float_to_str(capacitance_value, temp, 3)

        // Display capacitance value in correct units
        IF capacitance_value < 1.0 THEN
            Call snprintf(str, "%s uF", temp)
        ELSE IF capacitance_value < 1000.0 THEN
            Call snprintf(str, "%s uF", temp)
        ELSE
            Set mF_value to capacitance_value / 1000
            Call float_to_str(mF_value, temp, 3)
            Call snprintf(str, "%s mF", temp)
        END IF

        // Show result on screen
        Call DisplayCenteredTextInRectangle()
    END IF
END IF
```

2.2.4.4 User interface



The figure above shows the user interface of the digital multimeter (DMM) displaying the measured capacitance value of 19.103 µF after the user selects the Capacitance function.

2.2.4.5 Algorithm



The capacitance is calculated by:

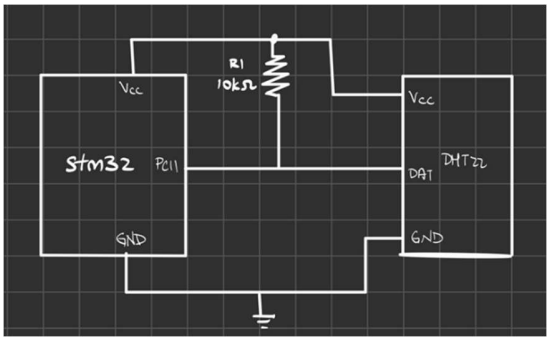
- Measuring the times it takes for the capacitor to charge to a threshold voltage.
- Using the resistor value and a constant derived from the threshold voltage ratio to compute the capacitance value.

2.2.5 Temperature measurement (DHT22 digital sensor).

2.2.5.1 Functionality / Feature

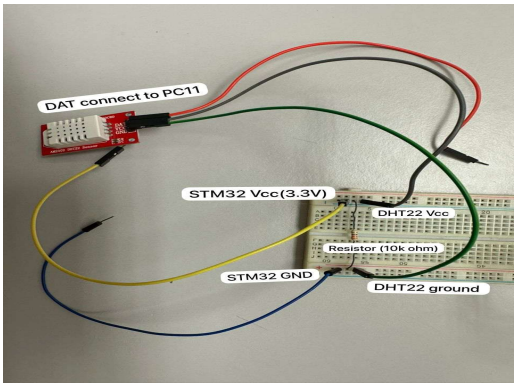
- Measure temperature using DHT22 sensor**.**
- Display temperature on an LCD screen with one decimal place**.**

2.2.5.2 Hardware Architecture



This figure shows the connection between an STM32 and a DHT22 sensor

- Microcontroller: STM32 microcontroller, with the data line of sensor connected to PC11 pin.
- Sensor: DHT22 (temperature sensor).
- Resistor: A 10kΩ resistor placed between VCC and DATA line of the DHT22 to ensure proper communication.
- Power Supply: The DHT22 is powered via the VCC pin (3.3V)
- Grounding: Connect DHT22 to microcontroller ground (GND)



This image shows the physical connection of the DHT22 sensor with the necessary components.

2.2.5.4 Software architecture (pseudocode)

```
// Step 1: Check if the correct button (DMM Temperature) is pressed
if button_id equals dmm_temp_button_id then

// Step 2: De-initialize previous DMM modes to clear settings
Call DMM_CAP_Deinit()   // De-initialize capacitance measurement
Call DMM_VOL_Deinit()   // De-initialize voltage measurement
Call DMM_TEMP_Deinit()  // De-initialize temperature measurement (if previously initialized)
Call DMM_RES_Deinit()   // De-initialize resistance measurement

// Step 3: Set the system into temperature measurement mode
Set cur_mode to DMM_MODE  // Change the current mode to Digital Multimeter (DMM) mode
Set DMM_BUTTON_SEL to DMM_TEMP_MODE  // Set the active button selection to temperature mode

// Step 4: Declare variables to store temperature and its parts
Declare Temp as float  // Variable to store the complete temperature value (Celsius)
Declare Temp_int as integer  // Integer part of the temperature (whole degrees)
Declare Temp_frac as integer  // Fractional part of the temperature (one decimal place)
Declare aTextBuffer as string with length 200  // Buffer to store formatted temperature text

// Step 5: Initialize the DHT22 sensor and configure DMM for temperature
Call DHT22_Init()  // Initialize the DHT22 temperature and humidity sensor
Call DMM_TEMP_Config()  // Configure the system to measure temperature using the DMM

// Step 6: Read temperature data from the DHT22 sensor
Call DHT22_Read()  // Read temperature and humidity values from the DHT22 sensor

// Step 7: Extract the temperature value in Celsius from the DHT22 sensor
Set Temp to Call DHT22getTemperature()  // Retrieve the temperature value from the DHT22 sensor

// Step 8: Separate the integer and fractional parts of the temperature
Set Temp_int to (integer value of Temp)  // Extract the integer part (whole degrees) of the temperature
Set Temp_frac to (integer value of (Temp - Temp_int) * 10)  // Extract the fractional part (one decimal place)

// Step 9: Update button highlights without redrawing the entire display
```

```
Call DrawButtons()  // Update the button highlights on the screen without refreshing everything

// Step 10: Format the temperature as a string with one decimal place
Call sprintf(aTextBuffer, "%d.%d C", Temp_int, Temp_frac)  // Format temperature as "xx.x C"

// Step 11: Display the formatted temperature on the screen
Call DisplayCenteredTextInRectangle(
      DMM_DISPLAY_X,              // X position on the screen
      DMM_DISPLAY_Y,              // Y position on the screen
      DMM_DISPLAY_WIDTH,      // Width of the display area
      DMM_DISPLAY_HEIGHT,    // Height of the display area
      aTextBuffer,                  // The formatted temperature string to display
      &TM_Font_11x18,             // Font type to be used
      ILI9341_COLOR_WHITE       // Color of the text (white)
   )

end if                          // End of button press check
```

2.2.5.4 User interface

This image shows the display on the LCD when the Temperature button is pressed and the temperature data is successfully read and displayed correctly.

2.2.5.5 Algorithm

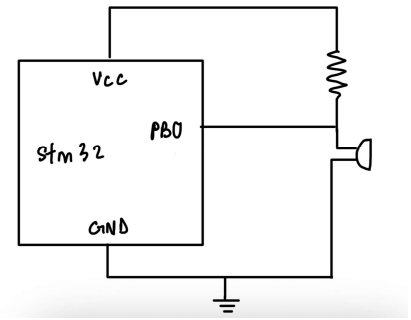
1. Initialize the DHT22 sensor and set up the LCD for display.
2. Read the temperature data from the DHT22 sensor.
3. Process the data to extract the temperature value.
4. Format the temperature to the correct display format.
5. Display the formatted temperature on the LCD screen.

2.2.6 Continuity test with buzzer activation when resistance ≤ 1Ω.

2.2.6.1 Functionality/ Feature

- Check if two points in a circuit are electrically connected
- A buzzer provides an audible alert when a connection is detected.
- The LCD displays “Connected” or “Unconnected” accordingly.

2.2.6.2 Hardware Architecture



This figure shows the connection between an STM32 and a buzzer.

- The hardware consists of an active buzzer connected to the STM32 microcontroller.
- GPIO pin PB0 is configured as a digital output to drive the buzzer.
- When PB0 outputs logic high, the buzzer is activated and generates sound.
- When PB0 outputs logic low, the buzzer is silent.
- A current-limiting resistor is connected in series with the buzzer to protect the circuit.
- Power is supplied from the system's VCC (3.3V) and GND.

2.2.6.3 Software Architecture (Pseudocode)

```
// Step 1: Check if the correct button (DMM Connectivity) is pressed

if button_id equals dmm_cont_button_id then

// Step 2: De-initialize previous DMM modes
Call DMM_CAP_Deinit()   // De-initialize capacitance mode
Call DMM_VOL_Deinit()   // De-initialize voltage mode
Call DMM_TEMP_Deinit()  // De-initialize temperature mode
Call DMM_RES_Deinit()   // De-initialize resistance mode

// Step 3: Set the system into connectivity measurement mode
Set cur_mode to DMM_MODE
Set DMM_BUTTON_SEL to DMM_CONT_MODE

// Step 4: Configure resistance measurement hardware
Call DMM_RES_Config()

// Step 5: Measure resistance value
Declare read_value as float
Set read_value = Call resisMeasure()

// Step 6: Update button highlights
Call DrawButtons()

// Step 7: Check resistance threshold and determine connection status
if read_value >= 0.0 AND read_value <= 1.0 then
   Call Buzzer_Control(1)   // Turn buzzer ON
```

```
   Display "Unconnected" in RED on LCD
else
   Call Buzzer_Control(0)   // Turn buzzer OFF
   Display "Connected" in BLUE on LCD
end if

end if                          // End of button press check
```

2.2.6.4 User Interface

This image shows the display on the LCD when the Connectivity button is pressed and displayed whether it is connected/ disconnected.

2.2.6.5 Algorithm

1. Read resistance value between two probes using the ADC and bridge circuit.
2. Compare resistance value to see whether between 0-1Ω.
3. If resistance is between, it means the probes are disconnected, turn buzzer ON and display “Unconnected”.
4. If resistance is more than 1, it means the probes are connected, turn buzzer OFF and display “Connected”.

2.3 Other Features

- Splash screen (logo display for 3s at startup).

- Switch between oscilloscope and DMM modes on the fly.

- The oscilloscope display allows the user to select between two channels, save or call waveform data, and return to the main menu.

- The display shows the DMM functions menu where the user can select from options for measuring Temperature, Current, Voltage, Resistance, Capacitance, Connectivity and can also return to the Main menu.

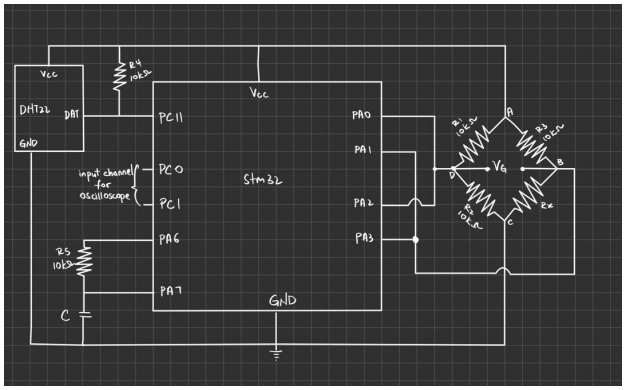
2.4 Overall

2.4.1 Functionality / Feature

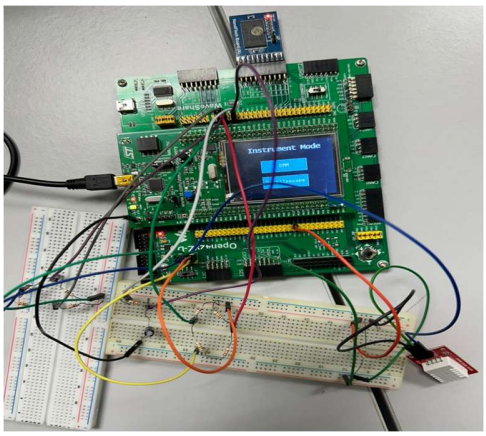
2-in-1 Measurement Instrument with STM32:

- Dual-Channel Oscilloscope for capturing and displaying signals, measuring frequency, duty cycle, and time base, with waveform storage and recall.
- Digital Multimeter (DMM) for measuring voltage, current, temperature, capacitance, resistance, and connectivity.
- Allows switching between oscilloscope and DMM modes with selectable DMM functions.

2.4.2 Hardware Architecture

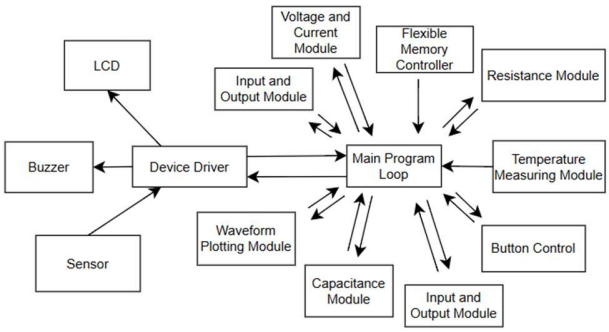


This figure shows the design of a system that integrates an oscilloscope and a digital multimeter (DMM) using an STM32 microcontroller.



This figure shows the physical setup of the instrument.

2.4.3 Software Architecture



- The main program loop controls and coordinates all modules.
- The voltage and current module measure electrical voltage and current.
- The resistance module measures resistance values.
- The capacitance module measures capacitance.
- The temperature measuring module reads temperature from the sensor.
- The input and output module manages user input and output signals.

- The button control detects and processes button presses.
- The flexible memory controller stores and recalls measurement data.
- The waveform plotting module displays waveforms on the LCD.
- The device driver links software modules to hardware devices.
- The LCD shows results, menus, and waveforms.
- The buzzer provides sound feedback, such as for continuity.
- The sensor supplies raw data for measurements like temperature or connectivity.

2.4.4 Algorithm

1. Start the system and display splash screen for 3 seconds.
2. Initialize hardware modules (ADC, LCD, keypad/buttons, buzzer, memory, sensors).
3. Enter the main program loop.
4. Wait for user input to select between DMM mode or Oscilloscope mode.

If DMM mode is selected:

5. Show DMM menu (Voltage, Current, Resistance, Capacitance, Temperature, Connectivity).
6. Read button/key input to choose measurement type.
7. Perform the selected measurement:

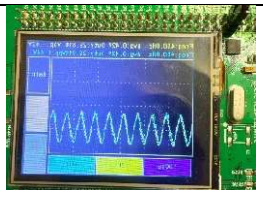
- Voltage → read ADC via voltage divider.
- Current → read ADC via shunt resistor amplifier.
- Resistance → use Wheatstone bridge circuit and calculate value.
- Capacitance → measure charge/discharge timing or divider method.
- Temperature → read sensor (e.g., DHT22).
- Connectivity → check resistance near 0 Ω and activate buzzer.
- Display the measurement result on LCD.
- Return to DMM menu for new selection.

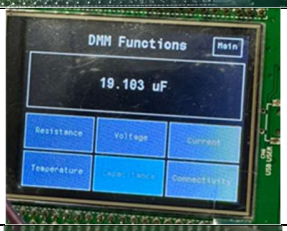
If Oscilloscope mode is selected:

8. Acquire ADC data from two channels.
9. Process signals (frequency, average, duty cycle, peak-to-peak).
10. Plot waveform on LCD with grid and selectable time base.
11. Provide options to store or recall waveform from memory.
12. Return to oscilloscope menu for new action.

13. Repeat the main loop until system is powered off.

3.0 Test and Verification

Test Case	Verification	Result
Oscillator	Correct — The 1.5V voltage value read correctly it is working as intended. During testing, the Frequency 450 Hz match to the time base 100ms/div. The 0.43V average voltage is correct. Duty cycle 30% is correct. The waveform also plotted follow the grid scale.	

Temperature	Correct — The temperature measurement function is working as intended. During testing, the system displayed a temperature of 36.1 °C, which matched the reference thermometer reading within the acceptable tolerance range.	
Capacitance	Correct — The capacitance measurement function is working as intended. During testing, the system displayed a capacitance of 19.103 µF, which is within the acceptable tolerance range for a 10 µF capacitor (±20%).	
Current	Wrong — The current measurement function is working but during testing, the system displayed a current of 0.6 mA, which is not expected this could be due to slight measurement errors or drift in the system.	
Voltage	Correct — The voltage measurement function is working. During testing, the system displayed a voltage of 0.0 V, which matched the Wheatstone Bridge voltage (V_G) result.	
Resistance	Correct — The resistance measurement function is working. During testing, the system displayed a resistance of 10000.0 Ω, which closely matched the reference calculation 10 kΩ resistor value.	
Connectivity	Fail — The connectivity (continuity) measurement function is not working. During testing, the system still activated the buzzer and displayed “Connected” when the circuit when the circuit was open.	

4.0 Future Improvement

For oscilloscope:

- Implement zoom and pan on the waveform display
- Improve the measurement algorithms
- Add triggered options like rising edge, falling edge, level trigger for stable waveform capture

For Digital Multi meter

- Expand measurement ranges, higher voltage, current and resistance
- Enhance the measurement into real-time measuring, the value measured can keep changing
- Add additional measurement functions like inductance, diode test

5.0 Conclusion

The proposed 2-in-1 measurement instrument, developed using an STM32 development board, integrates both Oscilloscope and Digital Multimeter (DMM) functionalities into a single platform. The system supports the measurement of a wide range of electrical parameters, including voltage, current, resistance and capacitance. The oscilloscope module utilizes dual ADC channels to capture and display signals in real time, providing key measurements such as frequency, average value, duty cycle and peak-to-peak voltage. In parallel, the DMM module enables precise measurement of voltage, current and capacitance.

Several issues were encountered during system development. The temperature measurement lacks continuous updates, reducing its effectiveness for real-time monitoring. The resistance measurement method is inaccurate, which leads to errors in the corresponding voltage and current calculations. Moreover, the oscilloscope functionality does not support square wave signals, limiting its capability for certain waveform analyses. Even with these errors, the system was completed and at least a working prototype was produced.

5.0 Reference

[1] R. K. Gupta, A. R. Hegde, and P. A. Thakar, Embedded systems: Design and applications using STM32, McGraw-Hill Education, 2015.

[2] Keysight Technologies, Oscilloscope fundamentals. [Online]. Available: https://www.keysight.com. [Accessed: Sep. 2, 2025].

[3] NXP Semiconductors, Capacitance measurement with STM32 microcontrollers. [Online]. Available: https://www.nxp.com. [Accessed: JUL. 22, 2025].

[4] D. M. W. Smith and B. P. M. Vaessen, Practical electronics for inventors, 4th ed., McGraw-Hill Education, 2016.

[5] J. S. Smith, The scientist and engineer's guide to digital signal processing, California Technical Publishing, 2008.

[6] Texas Instruments, Digital multimeter (DMM) design guide. [Online]. Available: https://www.ti.com/lit/an/slva517/slva517.pdf. [Accessed: Sep. 22, 2025].

[7] STM32 Microcontroller Documentation, STMicroelectronics. [Online]. Available: https://www.st.com/en/microcontrollers/stm32. [Accessed: Sep. 22, 2025].